

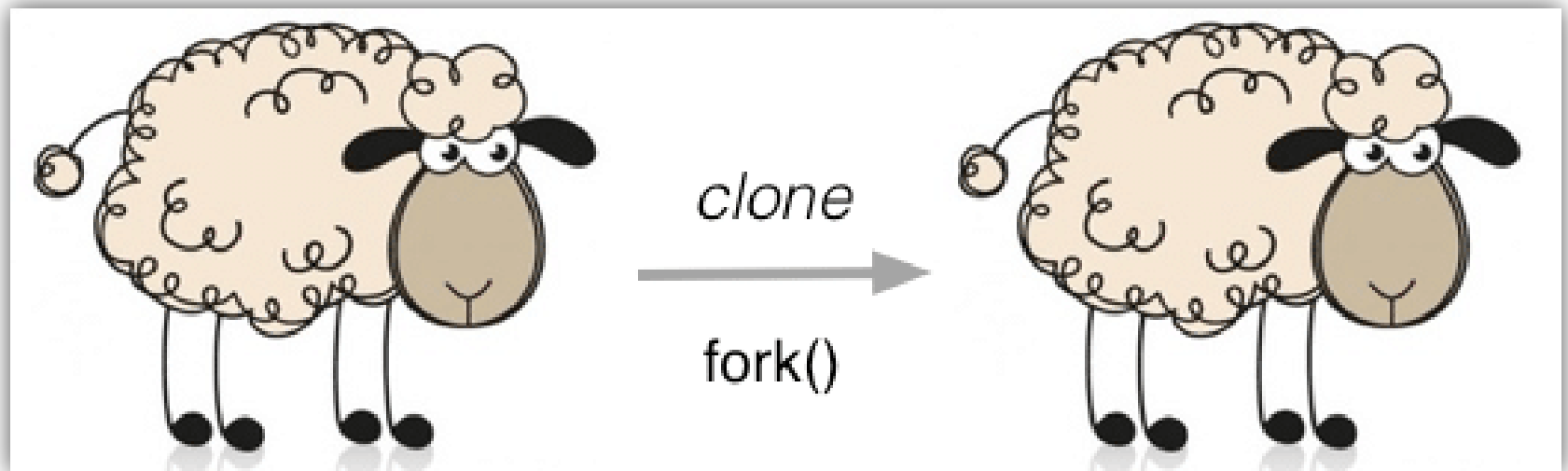
Sistemas Operacionais

Comunicação Entre Processos



FORK

(Criando Processos)



Criando Processos (arquivo moodle)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/types.h>
4 #include <unistd.h>
5
6 int main(void)
7 {
8     int i;
9     pid_t pid;
10
11     if ((pid = fork()) < 0)
12     {
13         perror("Erro na Criação do fork");
14         exit(1);
15     }
16     if (pid == 0)
17     {
18         //0 código no processo filho
19         int x = 0;
20         while (x < 100)
21         {
22             printf("-PID do Filho: %d - Execução: %d\n", getpid(), x);
23             x++;
24             sleep(1);
25         }
26     }
27     else
28     {
29         //0 código no processo pai
30         int x = 0;
31         while (x < 100)
32         {
33             printf("+PID do Pai: %d - Execução: %d\n", getpid(), x);
34             x++;
35             sleep(2);
36         }
37     }
38
39     printf("Processos Finalizados - Area comum aos processos. \n Digite um valor e Pressione enter:");
40     scanf("Voce Digitou %d", &i);
41     exit(0);
42 }
43
```

Criando Processos

Realizando o FORK (arquivo moodle)

```
01575855062@aula: ~  
01575855062@aula:~$ nano fork.c  
01575855062@aula:~$ gcc fork.c -o appfork  
01575855062@aula:~$ ls -la  
total 64  
drwxr-xr-x 5 marcelasantos Domain Users 4096 Aug 25 14:41 .  
drwxr-xr-x 3 root root 4096 Aug 25 13:15 ..  
-rwxr-xr-x 1 marcelasantos Domain Users 16960 Aug 25 14:41 appfork  
-rw-r--r-- 1 marcelasantos Domain Users 220 Aug 25 13:15 .bash_logou  
t  
-rw-r--r-- 1 marcelasantos Domain Users 3771 Aug 25 13:15 .bashrc  
drwx----- 2 marcelasantos Domain Users 4096 Aug 25 13:15 .cache  
drwx----- 4 marcelasantos Domain Users 4096 Aug 25 13:29 .config  
-rw-r--r-- 1 marcelasantos Domain Users 808 Aug 25 14:33 fork.c  
drwxr-xr-x 3 marcelasantos Domain Users 4096 Aug 25 13:40 .local  
-rw-r--r-- 1 marcelasantos Domain Users 18 Aug 25 13:39 nano  
-rw-r--r-- 1 marcelasantos Domain Users 807 Aug 25 13:15 .profile  
-rw----- 1 marcelasantos Domain Users 2837 Aug 25 13:39 .viminfo  
01575855062@aula:~$ ./appfork  
+PID do Pai: 636227 - Execucao: 0  
-PID do Filho: 636228 - Execucao: 0  
-PID do Filho: 636228 - Execucao: 1  
+PID do Pai: 636227 - Execucao: 1  
-PID do Filho: 636228 - Execucao: 2  
-PID do Filho: 636228 - Execucao: 3
```

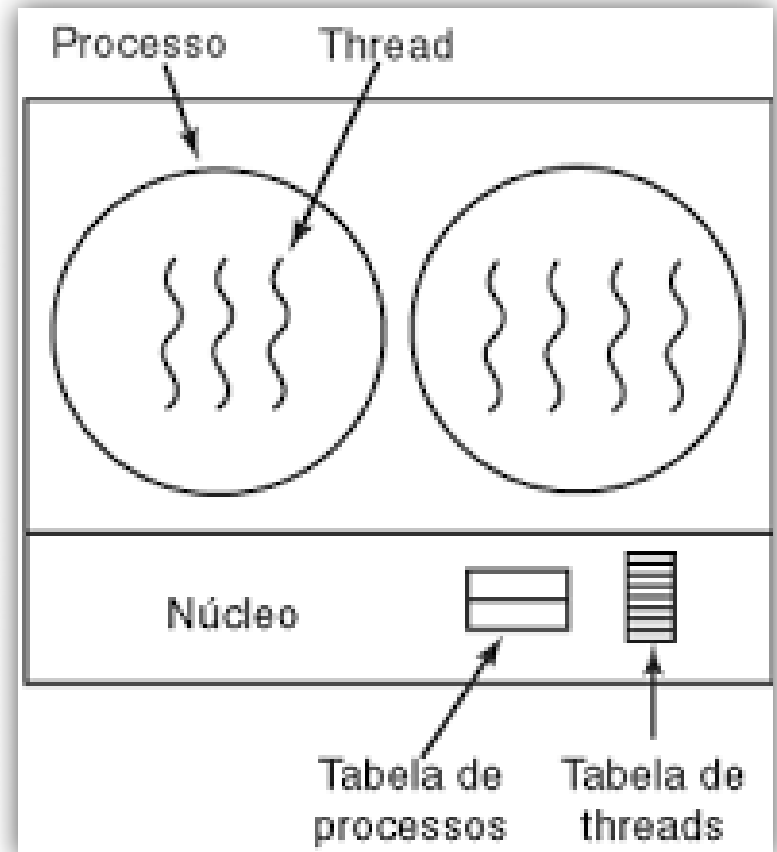
Criando Processos

Realizando o FORK (arquivo moodle)

```
01575855062@aula: ~  
-PID do Filho: 636256 - Execucao: 01575855062@aula:~$ ps -aux | grep app  
-PID do Filho: 636256 - Execucao: marcel+s+ 636255 0.0 0.0 2488 516 pts/2 S+ 14:43 0:00 ./appfork  
+PID do Pai: 636255 - Execucao: marcel+s+ 636256 0.0 0.0 2488 76 pts/2 S+ 14:43 0:00 ./appfork  
-PID do Filho: 636256 - Execucao: marcel+s+ 636271 0.0 0.0 6300 720 pts/1 S+ 14:43 0:00 grep --color=auto app  
-PID do Filho: 636256 - Execucao: 01575855062@aula:~$ ps -aux | grep app  
+PID do Pai: 636255 - Execucao: marcel+s+ 636255 0.0 0.0 2488 516 pts/2 S+ 14:43 0:00 ./appfork  
-PID do Filho: 636256 - Execucao: marcel+s+ 636256 0.0 0.0 2488 76 pts/2 S+ 14:43 0:00 ./appfork  
-PID do Filho: 636256 - Execucao: marcel+s+ 636277 0.0 0.0 6300 656 pts/1 S+ 14:43 0:00 grep --color=auto app  
+PID do Pai: 636255 - Execucao: 01575855062@aula:~$ █  
-PID do Filho: 636256 - Execucao: █  
-PID do Filho: 636256 - Execucao: █  
+PID do Pai: 636255 - Execucao: █  
-PID do Filho: 636256 - Execucao: █  
-PID do Filho: 636256 - Execucao: █  
+PID do Pai: 636255 - Execucao: █  
-PID do Filho: 636256 - Execucao: █  
-PID do Filho: 636256 - Execucao: █  
+PID do Pai: 636255 - Execucao: █  
-PID do Filho: 636256 - Execucao: █  
-PID do Filho: 636256 - Execucao: █
```

Threads

(Criando Fluxo de Execução)



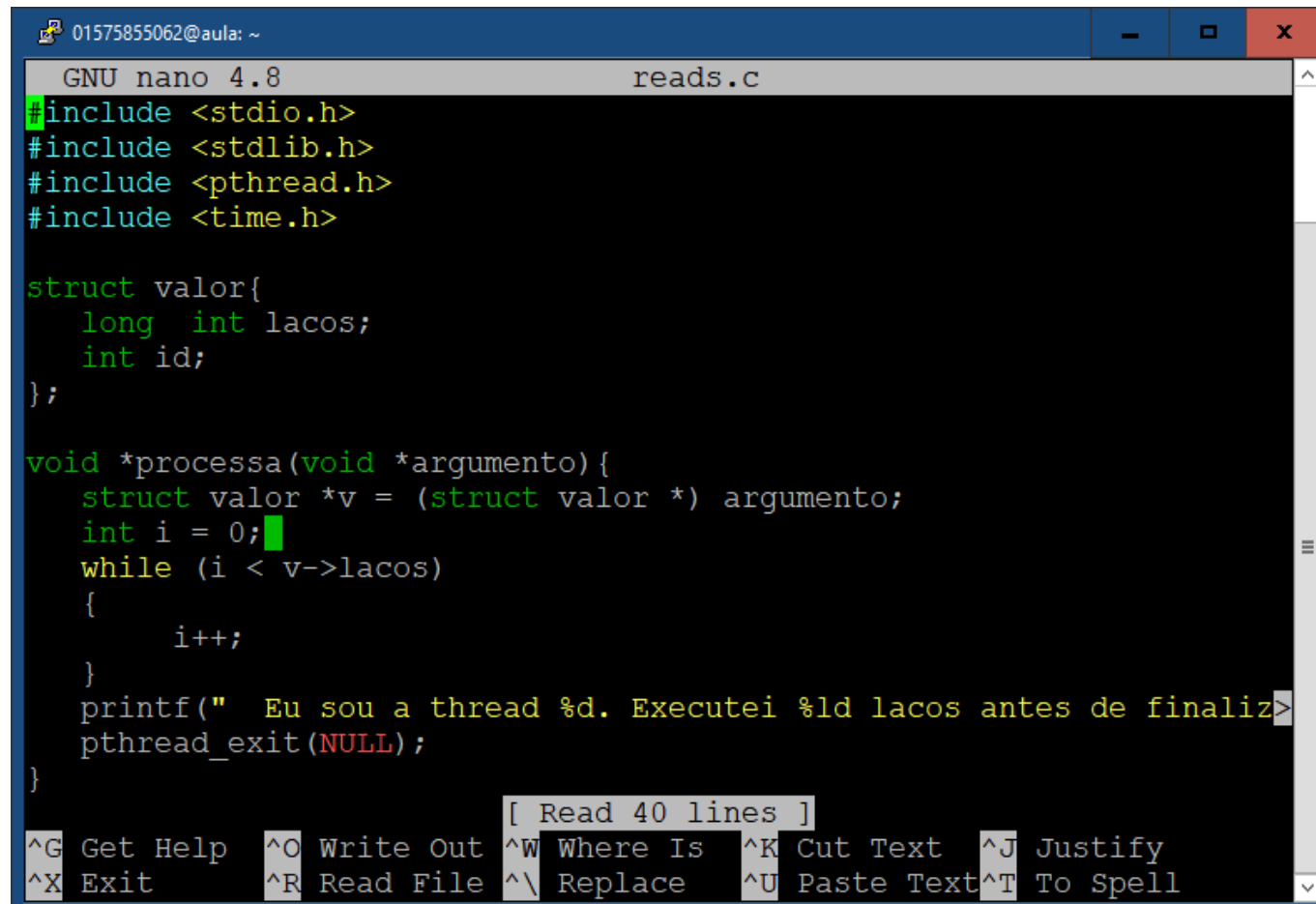


Criando Processos (arquivo moodle)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <time.h>
5  #include <unistd.h>
6
7  struct valor{
8      int lacos;
9      int id;
10 };
11
12 void *processa(void *argumento){
13     struct valor *v = (struct valor *) argumento;
14     int i = 0;
15     while (i < v->lacos)
16     {
17         i++;
18         sleep(1);
19     }
20     printf(" Eu sou a thread %d. Executei %d lacos antes de finalizar\n",v->id,v->lacos);
21     pthread_exit(NULL);
22 }
23
24 int main(){
25     pthread_t programas[10];
26     int execute,i;
27     struct valor *v;
28
29     srand(time(NULL));
30
31     for (i=0;i<2;i++){
32         v = (struct valor *) malloc(sizeof(struct valor *));
33         v->lacos = (200) ;
34         v->id = i;
35
36         printf("Criando a thread <%d> com <%d> lacos\n",i,v->lacos);
37         execute = pthread_create(&programas[i],NULL,processa,(void *)v);
38     }
39     pthread_exit(NULL);
40 }
41
```

Criando Processos

~\$ nano threads.c



```
01575855062@aula: ~
GNU nano 4.8 reads.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

struct valor{
    long int lacos;
    int id;
};

void *processa(void *argumento){
    struct valor *v = (struct valor *) argumento;
    int i = 0;
    while (i < v->lacos)
    {
        i++;
    }
    printf(" Eu sou a thread %d. Executei %ld lacos antes de finaliz>
    pthread_exit(NULL);
}

[ Read 40 lines ]
^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell
```


Criando Processos

Compilando & executando

~\$ gcc -o reads reads.c -lpthread

~\$./reads

```
01575855062@aula: ~
01575855062@aula:~$ gcc -o reads reads.c -lpthread
01575855062@aula:~$ ./reads
Criando a thread <0> com <200000000000> lacos
Criando a thread <1> com <200000000000> lacos
```

```
01575855062@aula: ~
1  [|||||||||||||||||||||100.0%]  Tasks: 55, 84 thr; 3 running
2  [|||||||||||||||||||||100.0%]  Load average: 1.45 1.22 1.62
3  [| 1.3%]  Uptime: 101 days(!), 05:29:20
4  [| 0.7%]
Mem[|||||||||476M/3.81G]
Swp[| 62.8M/3.82G]
```

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
639967	marcelsan	20	0	19140	1660	1532	R	99.8	0.0	1:15.83	./reads
639966	marcelsan	20	0	19140	1660	1532	R	99.2	0.0	1:15.76	./reads
639990	marcelsan	20	0	8064	3848	3224	R	1.3	0.1	0:00.31	htop
1333	mysql	20	0	1753M	27872	4460	S	0.0	0.7	3:35.05	/usr/sbin/mysqld
607316	www-data	20	0	195M	8536	3868	S	0.0	0.2	0:00.00	/usr/sbin/apache
607318	www-data	20	0	195M	8536	3868	S	0.0	0.2	0:00.01	/usr/sbin/apache
607320	www-data	20	0	195M	8560	3872	S	0.0	0.2	0:00.01	/usr/sbin/apache
607741	www-data	20	0	195M	8536	3868	S	0.0	0.2	0:00.01	/usr/sbin/apache
611136	www-data	20	0	195M	8532	3868	S	0.0	0.2	0:00.02	/usr/sbin/apache
632642	www-data	20	0	195M	8516	3808	S	0.0	0.2	0:00.00	/usr/sbin/apache
632645	www-data	20	0	194M	8140	3688	S	0.0	0.2	0:00.00	/usr/sbin/apache
633257	www-data	20	0	195M	8456	3808	S	0.0	0.2	0:00.00	/usr/sbin/apache
633258	www-data	20	0	194M	6600	2176	S	0.0	0.2	0:00.00	/usr/sbin/apache
633259	www-data	20	0	194M	6600	2176	S	0.0	0.2	0:00.00	/usr/sbin/apache

F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9Kill F10Quit

Criando Processos

Compilando & executando

~\$ gcc -o reads reads.c -lpthread

~\$./reads

```
lange@aula:~$ pstree
systemd--ModemManager--2*[{ModemManager}]
--VGAuthService
--accounts-daemon--2*[{accounts-daemon}]
--agetty
--apache2--10*[apache2]
--atd
--cron
--dbus-daemon
--firewalld--{firewalld}
--irqbalance--{irqbalance}
--multipathd--6*[{multipathd}]
--mysqld--29*[{mysqld}]
--networkd-dispat
--nscd--19*[{nscd}]
--nslcd--5*[{nslcd}]
--ntpd--{ntpd}
--polkitd--2*[{polkitd}]
--rsyslogd--3*[{rsyslogd}]
--snapd--14*[{snapd}]
--snmpd
--sshd--18*[sshd--sshd--sftp-server]
--9*[sshd--sshd--bash]
--9*[sshd--sshd--sh--bash]
--sshd--sshd--sh--bash--bash
--2*[sshd--sshd--bash--bash]
--3*[sshd--sshd--sh--sftp-server]
--sshd--sshd--bash--thread--2*[{thread}]
--2*[sshd--sshd]
--sshd--sshd--bash--pstree
--3*[systemd--(sd-pam)]
--systemd-journal
--systemd-logind
--systemd-network
--systemd-resolve
--systemd-udev
--udisksd--4*[{udisksd}]
--unattended-upgr--{unattended-upgr}
--upowerd--2*[{upowerd}]
--vmttoolsd--2*[{vmttoolsd}]

lange@aula:~$
```

Criando Processos

Compilando & executando

```
~$ gcc -o reads reads.c -lpthread
```

```
~$ ./reads
```

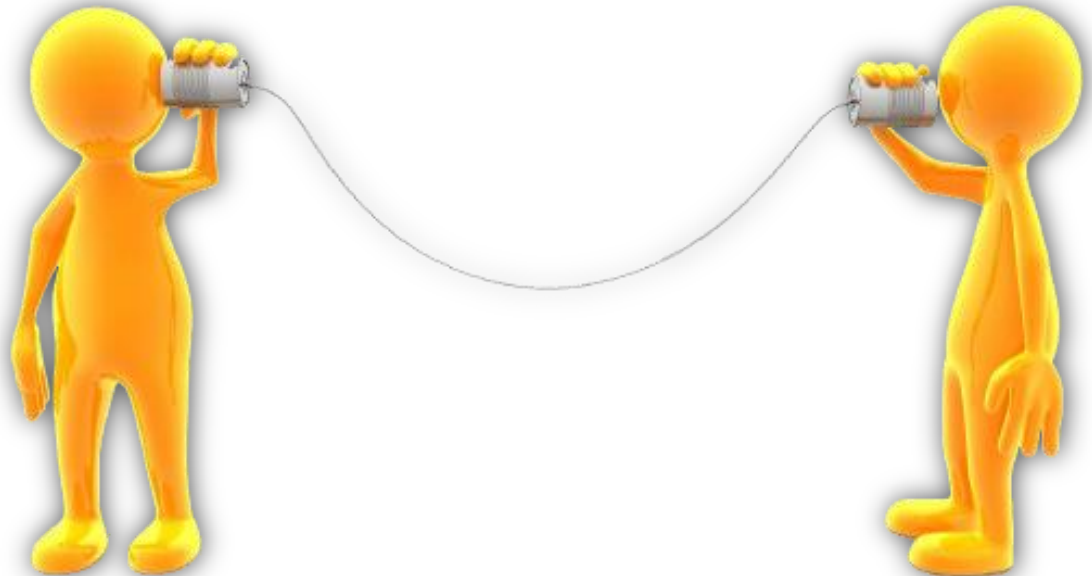
```
lange@aula:~$ ps -aux |grep thread
root          2  0.0  0.0   0      0 ?        S      Feb24   0:00 [kthreadd]
lange        664017  0.0  0.0   0      0 pts/29    Zl+    21:37   0:00 [thread] <defunct>
lange        664304  0.0  0.0  6432   720 pts/3     S+     21:39   0:00 grep --color=auto thread
lange@aula:~$
```

Criando Processos

Como medir o tempo de execução de uma tarefa?

~\$ time <nome do binário>

Comunicação entre Processos



Comunicação Entre Processos

- Durante a execução dos processos no Sistema Operacional, estes podem **ser independentes ou cooperativos**.
- A cooperação requer que os processos comuniquem entre si e **sincronizem** suas ações.

Comunicação Entre Processos

Principais motivos para cooperação entre processos

- Compartilhamento de informações
- Velocidade de computação
- Modularidade

Comunicação Entre Processos

Existem diferentes estratégias de comunicação entre processos: **IPC** (InterProcess Communication)

- Sockets (TCP/UDP)
- Fila de mensagens (*message queue*)
- PIPES (PID)
- Memória Compartilhada (Variáveis)

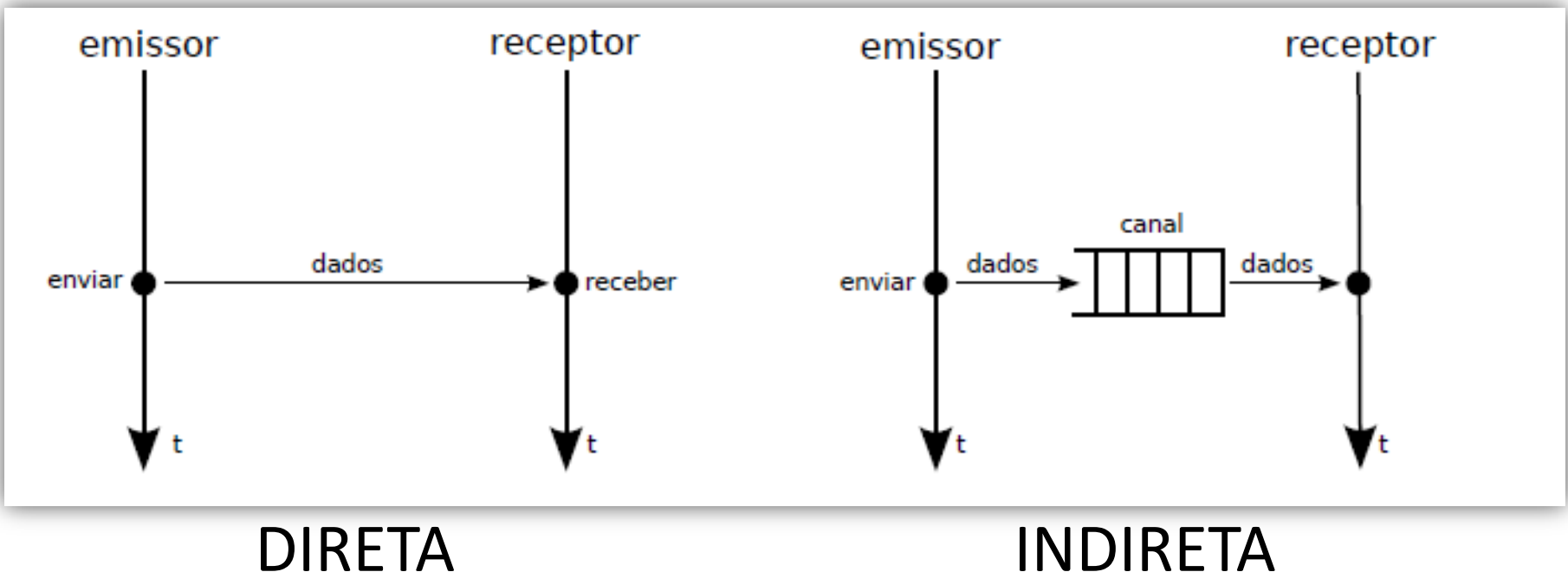
Comunicação Direta x Indireta

DIRETA

O emissor identifica claramente o receptor e vice-versa.

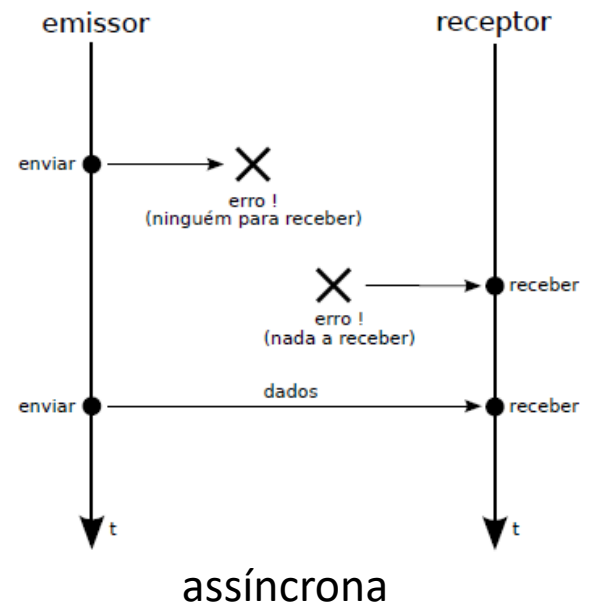
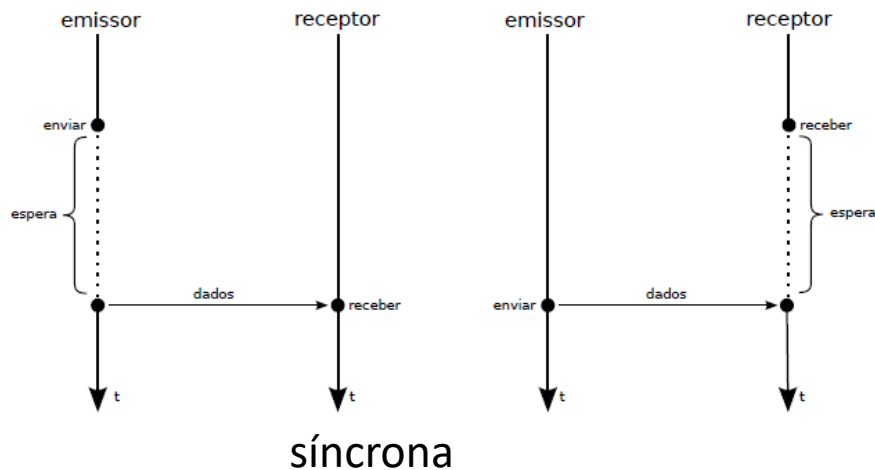
INDIRETA

O emissor e receptor não precisam se conhecer, pois não interagem diretamente entre si.



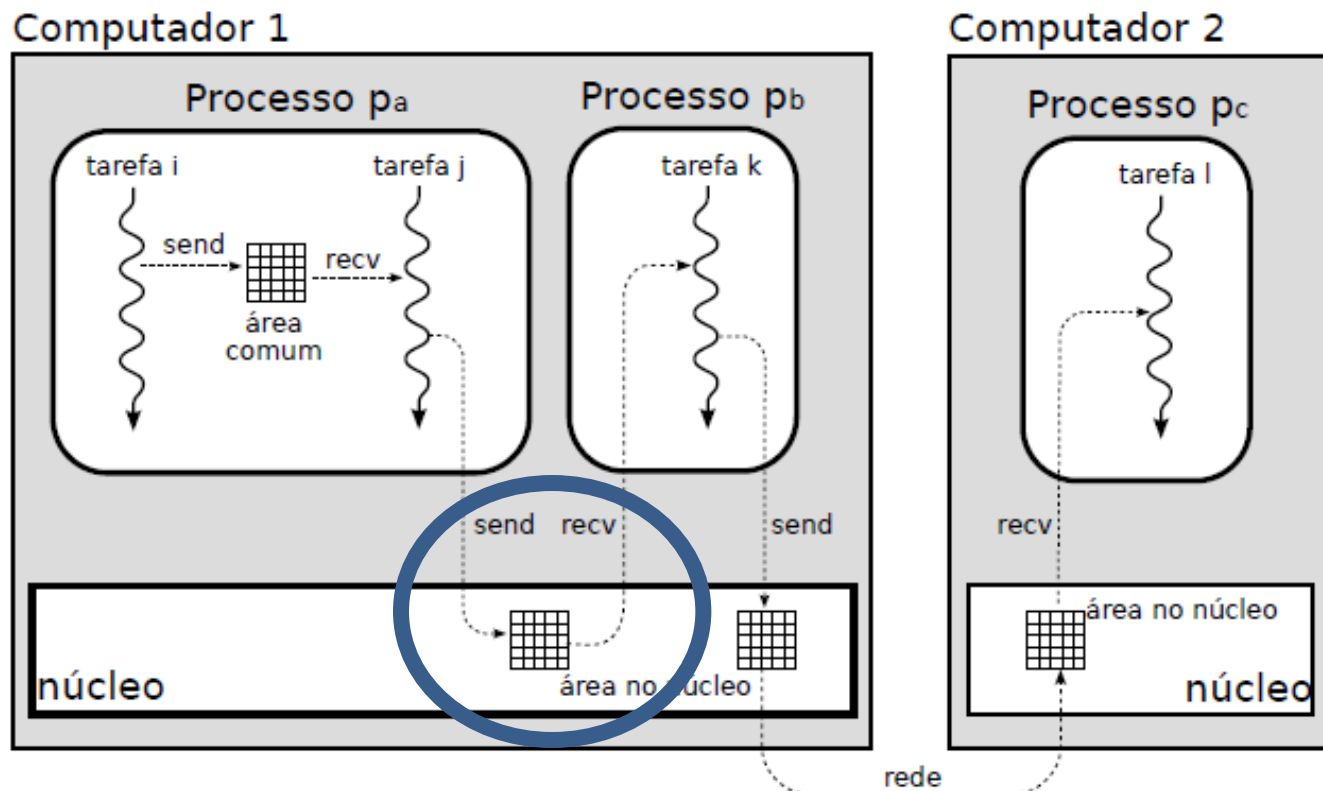
FILAS DE MENSAGENS

- O envio e recepção ordenada de mensagens *tipadas* entre processos locais.
- Envio e recepção podem ser síncronas ou assíncronas (escolha do programador)



FILAS DE MENSAGENS

- O produtor de mensagens deve ser executado após o consumidor, pois é este último quem cria a fila de mensagens
- As mensagens transitam apenas pela memória do núcleo



PIPES

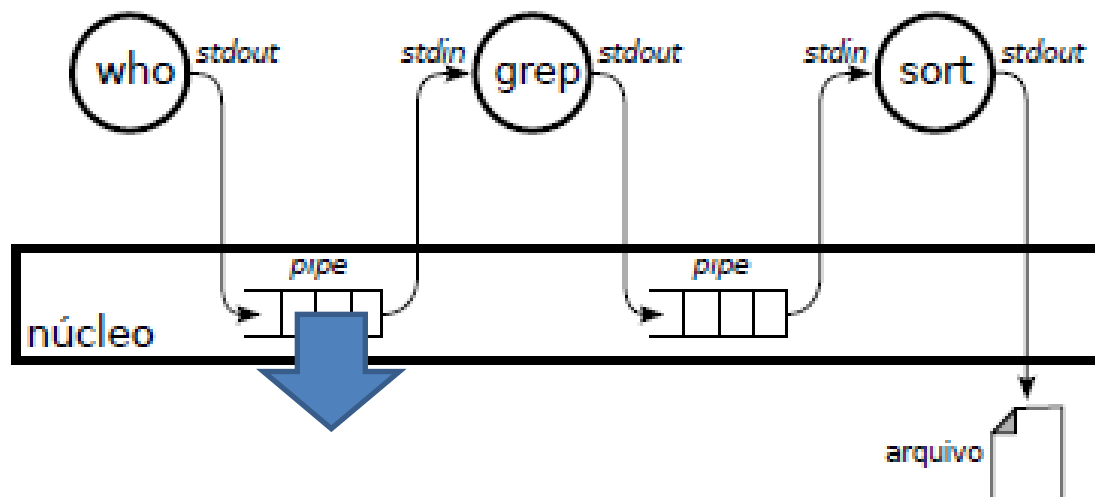
Muito utilizado para conectar stdout com stdin no shell

Exemplo:

pipe pipe



```
# who | grep ifrs | sort > login.txt
```



Buffer de 4KB no linux

MEMÓRIA COMPARTILHADA

- É gerenciado pelo núcleo
- Acesso ao conteúdo sem intermediação ou coordenação do núcleo
- Pode exigir mecanismos de coordenação para evitar condições de corrida.

SEÇÃO CRÍTICA

- Devem existir mecanismos para assegurar que somente um processo possa acessar um determinado recurso durante um certo tempo
- Estabelecem-se sessões de código que envolvem o recurso, denominadas seções críticas, e organiza-se o acesso a elas de tal modo que somente uma delas pode ser executada por um processo de cada vez

SEÇÃO CRÍTICA

Um processo impede que outros processos acessem as seções críticas que possui um determinado recurso compartilhado quando ele estiver acessando

Quando um processo finaliza a execução da seção crítica, outro processo pode executá-la

Mecanismo conhecido como **exclusão mútua**

SEÇÃO CRÍTICA

O Mecanismo mais simples de assegurar exclusão mútua para acesso a seções críticas é através de uma **FLAG**

A FLAG é uma variável de 1 bit que possui o valor 1 quando existe algum processo na seção crítica e 0 quando nenhum processo está na seção crítica

SEÇÃO CRÍTICA

Um processo que chega a seção crítica e a FLAG liberada pode entrar, bloqueando para prevenir que nenhum outro processo acesse.

Assim que o processo finaliza a execução da seção crítica, ele libera a FLAG e sai da sessão crítica.

SEÇÃO CRÍTICA - DEADLOCK

Pode ocorrer com dois processos quando um deles precisa de um recurso que está com outro e esse precisa de um recurso que está com o primeiro.

Exclusão mútua pode ser implementado utilizando **Semáforos**.

SEÇÃO CRÍTICA - DEADLOCK

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  pthread_mutex_t MeuSemaforo = PTHREAD_MUTEX_INITIALIZER;
5
6  int soma = 0;
7
8  void *contapar(void *param) {
9      int i;
10
11     // pthread_mutex_lock(&MeuSemaforo);
12
13     for (i = 0; i < 1000000; i++) {
14         soma += 2;
15     }
16     // pthread_mutex_unlock(&MeuSemaforo);
17     return NULL;
18 }
19
20
21 void *contaimpar(void *param) {
22     int i;
23
24     // pthread_mutex_lock(&MeuSemaforo);
25
26     for (i = 0; i < 1000000; i++) {
27         soma += 3;
28     }
29     // pthread_mutex_unlock(&MeuSemaforo);
30     return NULL;
31 }
32
33
34 int main() {
35     pthread_t tid1, tid2;
36     pthread_create(&tid1, NULL, contapar, NULL);
37     pthread_create(&tid2, NULL, contaimpar, NULL);
38
39     pthread_join(tid1, NULL);
40     pthread_join(tid2, NULL);
41
42     printf("A Soma das Threads eh %d\n", soma);
43     return 0;
44 }
```

```
gcc -o nome_saida nome_entrada.c -lpthread
```